A seguir está um plano de ação completo, estruturado para execução manual e pensado para ser consumido por um agente de IA focado em QA, considerando seu stack (Pytest + Selenium + Guará + GitHub Actions).

---

1. PLANO DETALHADO

1.1 Definir estratégia de gatilho (Trigger) no GitHub Actions

Objetivo: executar testes automaticamente em eventos relevantes

Passos:

1. Decidir os eventos que disparam a pipeline:
    - push em branch principal (main/master)
    - pull_request
    - release (principal objetivo do problema)
2. Configurar o workflow para rodar:
    - Em criação de release
    - Opcional: em tags (v*)

Resultado esperado:

- Pipeline dispara automaticamente sem intervenção manual

1.2 Criar estrutura do workflow no repositório

Objetivo: criar o arquivo de automação

Passos:

1. Criar diretório:

```
.github/workflows/
```

1. Criar arquivo:

```
tests.yml
```

1.3 Definir o ambiente de execução

Objetivo: preparar ambiente compatível com Selenium

Passos:

1. Definir runner:

```
runs-on: ubuntu-latest
```

1. Instalar dependências do sistema (browser + driver):

- Google Chrome
- ChromeDriver

1. Garantir suporte ao modo headless (obrigatório em CI)

1.4 Configurar instalação do Python e dependências

Objetivo: preparar o ambiente de testes

Passos:

1. Configurar Python 3.11:

```
- name: Setup Python
  uses: actions/setup-python@v4
  with:
    python-version: '3.11'
```

1. Instalar dependências:

```
- name: Install dependencies
  run: |
    python -m pip install --upgrade pip
    pip install -r requirements.txt
```

1. Validar instalação:

```
- name: Check pytest
  run: pytest --version
```

1.5 Configurar execução dos testes (Pytest)

Objetivo: executar suítes existentes (smoke, regression, e2e)

Passos:

1. Definir estratégia de execução:
    - Smoke → sempre em release
    - Regression → opcional (mais pesado)
    - E2E → controlado
2. Comando base:

```
pytest tests/specs --maxfail=1 --disable-warnings -v
```

1. Adicionar filtros:

```
pytest -m smoke
```

1. Configurar paralelismo (futuro):

```
pytest -n auto
```

1.6 Configurar evidências (logs, screenshots, relatórios)

Objetivo: gerar artefatos úteis

Passos:

1. Configurar saída de relatórios:

```
pytest --junitxml=reports/results.xml
```

1. Adicionar HTML report: Instalar:

```
pip install pytest-html
```

Executar:

```
pytest --html=reports/report.html --self-contained-html
```

1. Garantir captura de screenshots:

- Ajustar driver.py
- Em caso de falha:

Exemplo lógico:

```
def pytest_runtest_makereport(item, call):
    if call.when == "call" and call.excinfo is not None:

driver.save_screenshot("reports/screenshots/test_fail.png")
```

1.7 Upload de artefatos no GitHub Actions

Objetivo: disponibilizar evidências da execução

Passos:

1. Adicionar step:

```
- name: Upload reports
  uses: actions/upload-artifact@v3
  with:
    name: test-reports
    path: reports/
```

1. Incluir:

- report.html

- results.xml
- screenshots/

1.8 Configurar execução headless no Selenium

Objetivo: evitar falhas em ambiente CI

Passos:

1. Ajustar tests/fixtures/driver.py

Exemplo:

```
options = webdriver.ChromeOptions()
options.add_argument("--headless")
options.add_argument("--no-sandbox")
options.add_argument("--disable-dev-shm-usage")
options.add_argument("--window-size=1920,1080")
```

1. Garantir consistência:

- Mesmas opções local e CI (ou parametrizado via ENV)

1.9 Configurar variáveis de ambiente

Objetivo: desacoplar configurações

Passos:

1. Criar uso no GitHub Actions:

```
env:
  BASE_URL: "https://staging.app.com"
```

1. Consumir em settings.py:

```
import os

BASE_URL = os.getenv("BASE_URL")
```

1.10 Estratégia de separação de pipelines (recomendado)

Objetivo: melhorar performance e controle

Divisão sugerida:

Pipeline	Quando roda	Testes
CI	PR	Smoke

CD	Release	Smoke + Regression
Nightly	Agendado	E2E completo

1.11 Pipeline completo (exemplo pronto)

```yaml
name: Test Automation

on:
  push:
    branches: [ "main" ]

jobs:
  test:
    runs-on: ubuntu-latest

     env:
       BASE_URL: "https://staging.app.com"

     steps:
       - name: Checkout
         uses: actions/checkout@v3

       - name: Setup Python
         uses: actions/setup-python@v4
         with:
           python-version: '3.11'

       - name: Install Chrome
         run: |
           sudo apt-get update
           sudo apt-get install -y google-chrome-stable

       - name: Install dependencies
         run: |
           python -m pip install --upgrade pip
           pip install -r requirements.txt
           pip install pytest-html

       - name: Run tests
         run: |
```

```
36            mkdir -p reports/screenshots
37            pytest tests/specs \
38              --html=reports/report.html \
39              --junitxml=reports/results.xml \
40              --self-contained-html \
41              -m smoke
42
43        - name: Upload reports
44          uses: actions/upload-artifact@v3
45          with:
46            name: automation-reports
47            path: reports/
```

2. SUMÁRIO

O plano foi estruturado com base em uma abordagem profissional de QA focada em CI/CD:

1. Primeiro foi definido **quando executar os testes** (gatilhos)
2. Depois estruturado o **ambiente reprodutível (runner + Python + browser)**
3. Em seguida configurada a **execução dos testes com Pytest**
4. Criada a estratégia de **geração de evidências (HTML, XML, screenshots)**
5. Implementado o **upload de artefatos para análise**
6. Incorporadas boas práticas de:
    - Headless
    - Configuração por variáveis
    - Separação de pipelines

O plano garante:

- Execução automática confiável
- Observabilidade (relatórios)
- Escalabilidade futura

---

3. BOAS PRÁTICAS E MELHORIAS

3.1 Organização de testes

- Padronizar uso de markers:

```
1  @pytest.mark.smoke
```

```
@pytest.mark.regression
@pytest.mark.e2e
```

- Definir regras no pytest.ini

3.2 Estabilidade dos testes

- Implementar waits explícitos no Selenium
- Evitar sleep()
- Centralizar retry em operações críticas

3.3 Observabilidade

- Sempre gerar:
  - HTML report
  - XML (para integração com ferramentas)
  - Screenshot on failure

3.4 Performance

- Executar:
  - Smoke em PR
  - Regression em merge
  - E2E em nightly

3.5 Versionamento e qualidade

- Adicionar:
  - Lint (flake8)
  - Formatação (black)
  - type checking (mypy)

3.6 Integração com qualidade

- Futuro:
  - Allure Reports
  - Dashboard de testes
  - Integração com Jira/TestRail

3.7 Estratégia avançada (próximo nível)

- Matrix builds:

```
strategy:
  matrix:
    browser: [chrome, firefox]
```

- Paralelismo:

```
pytest-xdist
```

3.8 Manutenção do framework (Guará)

- Seguir rigorosamente:
  - Page Transactions Pattern
  - Separação Page vs Transaction
  - Reutilização de fixtures

Se quiser, posso evoluir esse plano para:

- Pipeline com Allure completo
- Execução paralela (pytest-xdist)
- Grid Selenium (Docker)
- Estratégia multi-ambiente (dev/staging/prod)
- Template reutilizável de workflow corporativo

Qual desses você quer implementar na próxima etapa?